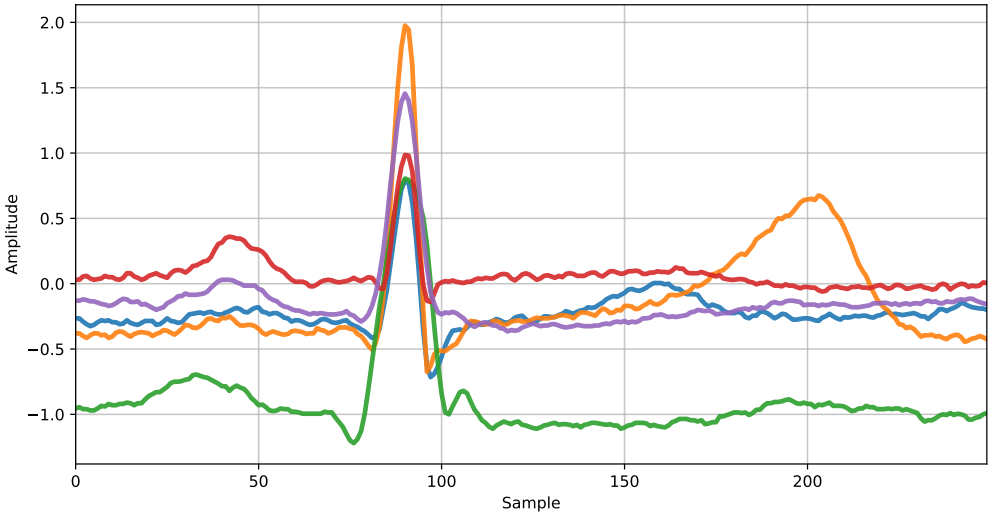
Delft University of Technology

Faculty of Electrical Engineering, Mathematics and Computer Science

Machine Learning for Electrical Engineering - EE4C12

ECG Classification for Arrhythmia Detection



Students

Marco Bertazzoli Cova Marco Fiorini

Summary

This project addresses ECG heartbeat classification into the five AAMI classes required by the assignment. The resulting dataset is strongly imbalanced.

We perform a data exploration procedure and we investigate normalization, standardization, PCA and FFT-based features. Trained models are selected using validation macro F1. The trained model families are logistic regression models, RBF SVM, fully connected neural networks and 1D convolutional neural networks.

The best-performing model on the validation set is RBF-SVM, with a macro F1 of approximately 0.939. The best neural network architecture is the one with two hidden layers, 100 neurons each, trained on the raw data. This achieves a validation macro F1 of 0.9375.

An additional experiment is conducted: a neural network with the best found architecture is trained for classifying all the 15 classes. This experiment highlights the difficulty of recognizing classes with very few training examples.

This project shows that greater architectural complexity does not necessarily improve performance. It also highlights the importance of evaluating minority classes alongside aggregate metrics.

1 Introduction and ML Pipeline

This project investigates ECG heartbeat classification into five AAMI classes and compares different machine learning approaches. The original labels are mapped to the AAMI groups, retaining the supplied training, validation and test splits. Preprocessing parameters and models are fitted using training data only. Preprocessing choices and hyperparameters are selected using validation macro F1. The selected SVM, MLP and CNN configurations are then evaluated on the test set. Validation and test data receive the preprocessing transformations established during training, following the workflow discussed in the course [1].

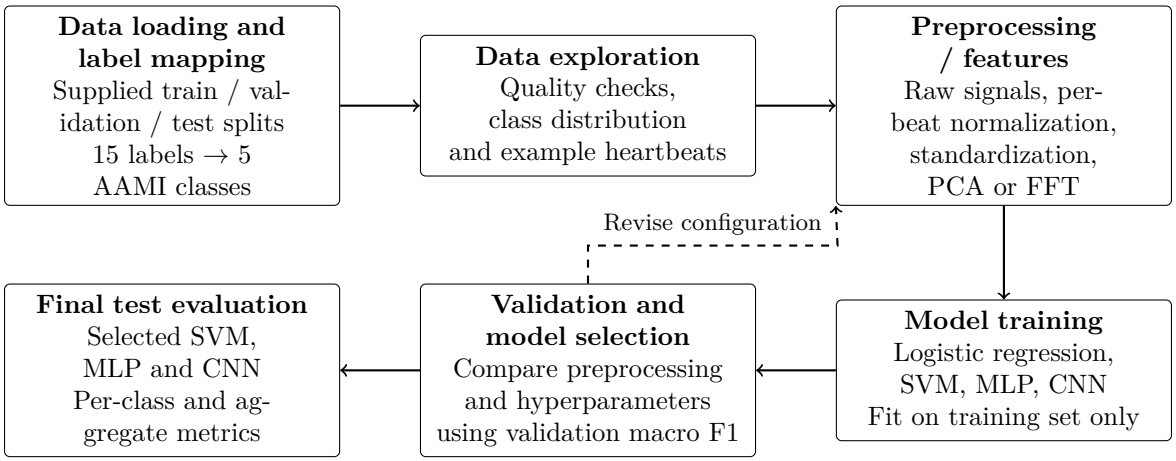


Figure 1: Project workflow. Preprocessing and feature representations are alternatives or combinations compared during validation, rather than a mandatory sequence. Any fitted transformation uses training data only and is applied unchanged to validation and test data. Dashed arrows indicate tuning; the test set is used after selection.

2 Task 1: Data Analysis and Feature Engineering

2.1 Data exploration

The supplied CSV files already contain segmented ECG heartbeats. Each row consists of 250 signal samples and one label identifying one of 15 original heartbeat types. The fixed splits contain 65,670 training beats, 21,891 validation beats and 21,891 test beats, approximately corresponding to a 60/20/20 split. Quality checks found no missing or infinite values in the training data.

The original labels are grouped into five AAMI classes, as reported in Table 1. The resulting dataset

remains strongly imbalanced: N accounts for 82.77% of training beats, whereas F accounts for only 0.73%. Always predicting N would therefore achieve 82.77% training accuracy without recognizing any minority class. This motivates per-class and macro-averaged evaluation.

AAMI class	Original beat types	Training beats	Share
N	N, L, R, e, j	54 355	82.77%
S	A, a, J, S	1 669	2.54%
V	E, V	4 341	6.61%
F	F	481	0.73%
Q	/, f, Q	4 824	7.35%

Table 1: AAMI mapping and training-set class distribution.

Some original types are particularly rare. Original label 9 (type S) has only two training examples and none in validation or test, preventing evaluation of its generalization performance in the 15-class experiment.

Figure 2 illustrates variations in baseline, amplitude and waveform shape within and between heartbeat types. These observations motivate investigating normalization and alternative feature representations.

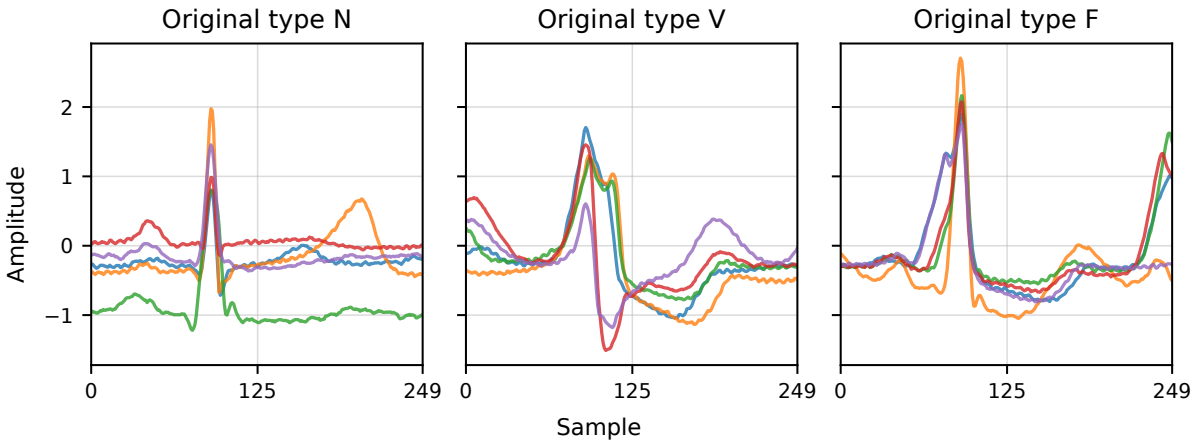


Figure 2: The first five training heartbeats from each original type N, V and F, plotted on common axes. These examples illustrate variability and do not represent the full distribution of each type.

2.2 Preprocessing methods

Since the supplied heartbeats are already segmented, we focus on scaling and feature representations. We compare per-beat normalization, standardization, PCA and FFT, assessing their usefulness through validation performance.

2.3 Standardization

All raw features are amplitude samples of the same ECG signal and therefore share a common measurement scale. Standardization was nevertheless tested to assess whether rescaling each input position could improve classification.

We applied standardization using `StandardScaler` [2]. The scaler is fitted on the training set and reused for validation and test data. For the tested two-layer MLP, standardization did not improve validation macro F1: it decreased from 0.9375 on raw signals to 0.9301 on standardized signals.

For logistic regression and SVM, we also tested per-beat normalization: each heartbeat is centered by subtracting its median and scaled by its standard deviation.

2.4 PCA analysis and dimensionality reduction

The 250 input features are correlated samples of the same heartbeat. PCA exploits this redundancy to reduce dimensionality, but preserving variance does not necessarily preserve features useful for classification. We examined cumulative explained variance, component shapes and per-class reconstruction errors to inform the choice of the number of retained components k.

2.4.1 Cumulative explained variance

PCA was fitted on raw training signals. Figure 3 shows that 5, 8 and 17 components retain 90%, 95% and over 99% of the variance, respectively. The first component alone explains 51.8%.

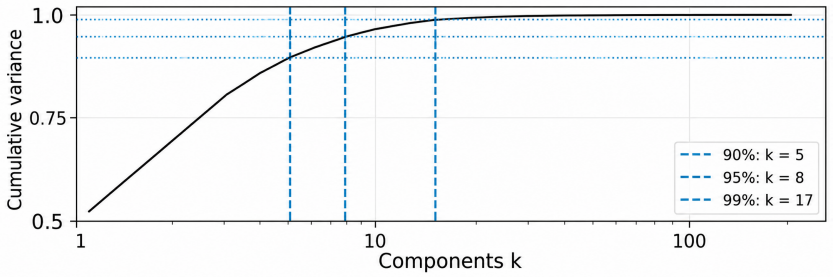


Figure 3: Cumulative explained variance of PCA fitted on the training set.

Although this result looks very promising, it does not mean that the remaining 1% of the variance is irrelevant. As noted above, the variance is dominated by normal beats, so the discarded components may contain exactly the details that distinguish the minority classes. This is investigated in Section 2.4.3.

2.4.2 Shape of the principal components

Each principal component is a vector of 250 weights, one for each time sample. It can therefore be plotted as a waveform and compared with the shape of a heartbeat. Figure 4 shows the first six components and the last three.

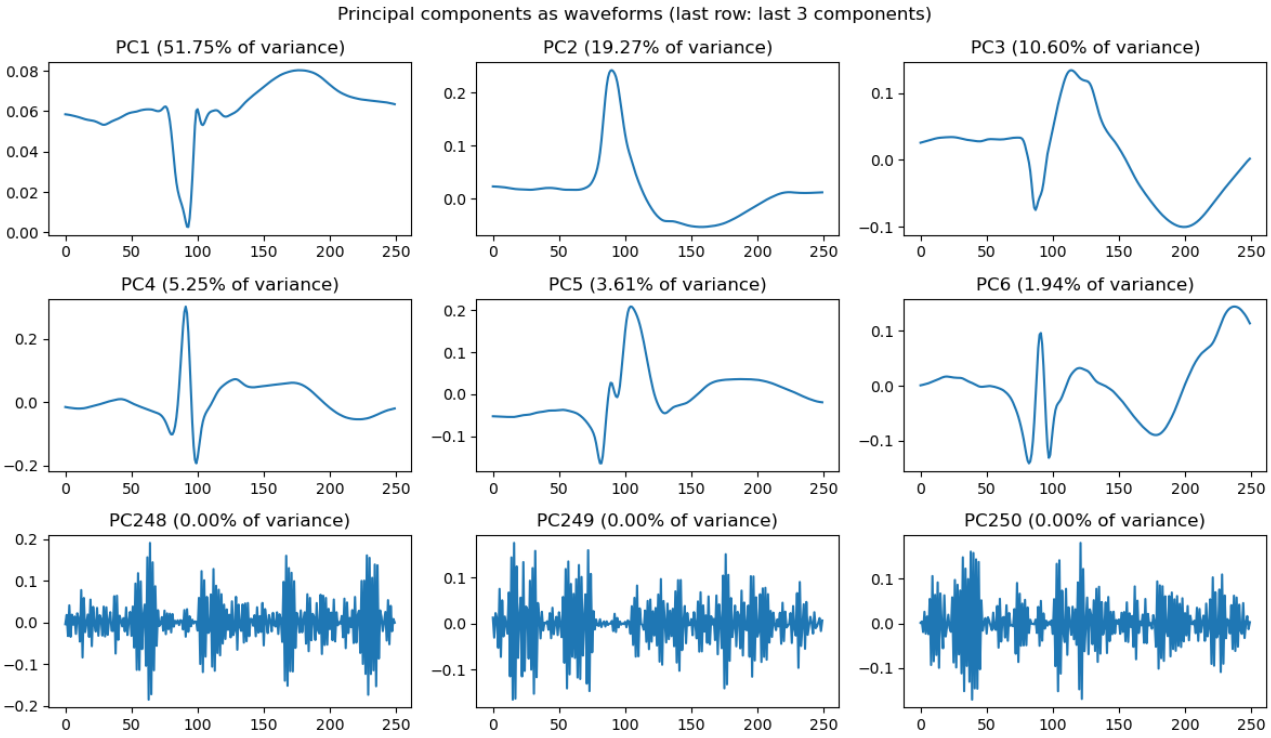


Figure 4: First six and last three principal components of the training set, plotted as waveforms over the 250 samples.

The first components are smooth and resemble parts of an ECG beat. The last components explain very little variance and show rapid oscillations. These may reflect noise or fine-scale signal details; their appearance alone does not establish whether they are useful for classification.

2.4.3 Per-class reconstruction error

To measure how much information is lost when only the first k components are kept, each beat $\mathbf{x}_i$ is projected onto these components and mapped back to the original 250 samples:

$$\hat{\mathbf{x}}_i = \boldsymbol{\mu} + \mathbf{W}_k \mathbf{W}_k^\top (\mathbf{x}_i - \boldsymbol{\mu}), \tag{1}$$

where $\boldsymbol{\mu}$ is the mean training beat and $\mathbf{W}_k \in \mathbb{R}^{250\times k}$ contains the first k principal components as columns. The reconstruction error of a beat is the mean squared difference between the original and the reconstructed signal:

$$e_i(k) = \frac{1}{250} \sum_{j=1}^{250} (x_{ij} - \hat{x}_{ij})^2 . \tag{2}$$

The errors are then averaged over the beats of each class c, giving $E_c(k)$. To compare the classes, we divide each value by the error of N, obtaining the relative error $R_c(k) = E_c(k)/E_N(k)$. A value of $R_c(k) = 3$ means that beats of class c are reconstructed three times worse than normal beats. PCA is fitted on the training set, and the errors are computed on the validation set.

The results are reported in Table 2. With $k = 17$, which retains 99% of the variance (Figure 3), S, V and Q are reconstructed about 2.7–2.8 times worse than N, and F about 1.9 times worse. These differences indicate uneven reconstruction across classes, but do not directly measure the classification value of discarded components.

k	N	S	V	F	Q
5	1.00	1.03	2.92	3.15	1.33
8	1.00	1.53	3.81	3.82	1.61
17	1.00	2.78	2.80	1.88	2.68
50	1.00	1.19	1.57	0.75	2.52
70	1.00	0.95	1.35	0.77	2.52
100	1.00	0.89	1.14	0.81	1.59
115	1.00	0.92	1.09	0.87	1.22

Table 2: Relative reconstruction error $R_c(k) = E_c(k)/E_N(k)$ of each class on the validation set, for different numbers of components k. Values are scaled on N, so the N column is always 1.

As k increases, the relative errors approach 1. From $k = 50$, S and F are reconstructed about as well as N and V is within a factor of 1.6, while Q still has a relative error of about 2.5. Only around $k = 100$ are all the classes within a factor of 1.6 of N.

With 100 components, reconstruction errors relative to N are closer to one, while dimensionality is reduced by 60%. However, reconstruction quality does not determine the number of components required for classification. For example, the MLP using 17 raw-signal PCA components achieved validation macro F1 of 0.9310, compared with 0.9375 on raw signals. The usefulness of PCA must therefore be assessed for each classifier using validation performance.

2.5 FFT-based feature representation

We investigated frequency-domain features as an alternative to the raw amplitude samples. Applying `np.fft.rfft` to each heartbeat produces 126 complex coefficients. We tested two representations: concatenating their real and imaginary parts, yielding 252 features, and retaining only their magnitudes, yielding 126 features.

```
spectrum = np.fft.rfft(X_train, axis=1)
# Real and imaginary parts: 252 features
```

```
X_train_fft = np.concatenate((spectrum.real, spectrum.imag), axis=1)
# Magnitudes only: 126 features
X_train_fft_mag = np.abs(spectrum)
```

Both representations were evaluated using the same MLP configuration, with two hidden layers of 100 neurons. Real and imaginary features achieved a validation macro F1 of 0.9373, close to the 0.9375 obtained on raw signals. Standardizing these features reduced the score to 0.9025.

Magnitude-only features reduced the input dimensionality by 49.6% compared with raw signals, but achieved a lower macro F1 of 0.9113. This representation discards phase information, which may contribute to the performance loss. The raw-signal model therefore remained the preferred MLP configuration.

3 Task 2: Model Development and Selection

3.1 Evaluation metrics and validation strategy

Class imbalance makes accuracy alone insufficient to assess classification performance: correct predictions for the majority class can conceal poor recognition of minority classes. We therefore use validation macro F1 as the main model-selection criterion. It averages the F1 scores of the five AAMI classes, giving each class equal weight.

We also report per-class precision, recall and F1. Precision measures the reliability of predictions for a class, while recall measures the fraction of its examples correctly identified. Accuracy and weighted F1 provide complementary aggregate measures. Confusion matrices show which classes are confused, with true labels in rows and predictions in columns.

We use the supplied fixed validation split to compare hyperparameters and feature representations. Final test results are reported for the selected configurations using the same evaluation metrics.

3.2 Logistic regression and trivial baseline

3.2.1 Trivial baseline

We used a majority-class baseline (`DummyClassifier` with `strategy="most_frequent"`) that always predicts N. On validation, it achieved 82.8% accuracy and macro F1 of 0.181, with zero recall for S, V, F and Q.

3.2.2 Softmax logistic regression

We use the `LogisticRegression` class of scikit-learn [2], which trains a softmax (multinomial) model on the five classes. For each class c it computes a linear score $z_c = \mathbf{w}_c^\top \mathbf{x} + b_c$ and turns the scores into probabilities:

$$p(y = c \mid \mathbf{x}) = \frac{\exp(z_c)}{\sum_{j=1}^{5} \exp(z_j)}. \tag{3}$$

The predicted class is the one with the highest probability, so the decision boundaries between classes are linear.

Since N accounts for 82.77% of the training beats, it would dominate the loss, and the model could largely ignore the minority classes. To avoid this, we set `class_weight="balanced"`, which weights each beat inversely to the size of its class. In this way, every class contributes the same total weight to the loss: a beat of class F, for example, weighs about 113 times more than a beat of class N. This should increase the recall of the minority classes, at the cost of more normal beats being classified as arrhythmias.

3.2.3 Preprocessing comparison and results

The classifier is kept fixed and only the preprocessing is changed. We compare four methods: raw signals (P0), per-beat normalization (P1), PCA with k components (P2) and PCA with whitening (P3), which also scales each kept component to unit variance. For each method, the inverse regularization strength $C \in \{10^{-3}, \ldots, 10^{2}\}$ and, for P2 and P3, the number of components $k \in \{17, 30, 50, 70, 100\}$

are selected by validation macro F1. The complete grid is reported in the notebook. Table 3 shows the best configuration of each method.

Method	C	k	Val. macro F1	Val. macro recall	Train macro F1
P1 per-beat normalization	0.1	–	0.554	0.834	0.552
P3 PCA + whitening	0.001	100	0.543	0.814	0.540
P0 raw	1	–	0.532	0.814	0.534
P2 PCA	1	100	0.531	0.813	0.532
Trivial baseline	–	–	0.181	0.200	–

Table 3: Best configuration of each preprocessing method for logistic regression, selected by validation macro F1.

All methods reach about three times the macro F1 of the trivial baseline, but the differences between them are small. Per-beat normalization gives the best result, with a gain of only 0.022 over raw signals.

For logistic regression, validation macro F1 increased from 0.474 with 17 PCA components to 0.531 with 100 components, close to 0.532 on raw signals. Among the tested settings, 100 components therefore came closest to the raw-input score. This result is specific to this classifier. Whitening performed slightly better than plain PCA, with strong regularization ($C = 0.001$).

Finally, training and validation macro F1 differ by less than 0.005 for every method. The model does not overfit, it underfits: the limit might be the linear decision boundary.

3.3 Support Vector Machine

Logistic regression reached a validation macro F1 of only 0.554, with almost the same score on the training set (Table 3). A linear SVM would face the same problem, since it also separates the classes with hyperplanes and only the loss function changes. We therefore use an SVM with a radial basis function (RBF) kernel,

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right), \tag{4}$$

which measures the similarity between two beats. A new beat is classified through a weighted sum of its similarities to the support vectors, which gives a non-linear decision boundary.

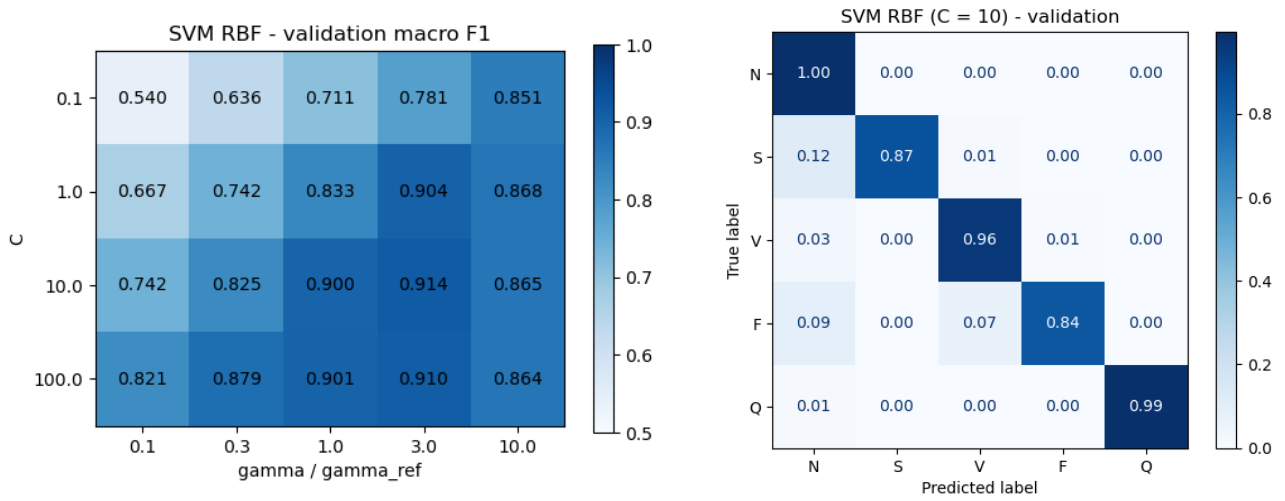
We use the `SVC` class of scikit-learn [2] with an RBF kernel and, as for logistic regression, balanced class weights. For the five classes, `SVC` trains one binary SVM for every pair of classes and predicts by majority vote. The two hyperparameters are C, the cost of a margin violation, and γ, the inverse width of the kernel. Large values of either give a more complex boundary and risk overfitting, while a small γ gives a smooth boundary, close to a linear one.

For the SVM, we combined per-beat normalization with PCA using 100 components. Normalization was motivated by its performance with logistic regression, while dimensionality reduction lowered the computational cost of kernel evaluations. Here, PCA was fitted on normalized training signals, whereas the earlier reconstruction analysis used raw signals. The retained components explain 99.92% of the variance after normalization.

Since a full grid search on all 65 670 training beats would take hours, we tuned the hyperparameters on a stratified 20% subset of the training set (13 134 beats) and scored every combination on the full validation set. The grid is $C \in \{0.1, 1, 10, 100\}$ and $\gamma = f \cdot \gamma_{\text{ref}}$ with $f \in \{0.1, 0.3, 1, 3, 10\}$, where $\gamma_{\text{ref}} = 0.0077$ is the default value used by scikit-learn (`gamma="scale"`).

Figure 5a shows the validation macro F1 of the grid. With small C and γ the score is 0.540, the same as logistic regression: with a wide kernel, the SVM behaves almost like a linear model. The score increases with γ up to $3\gamma_{\text{ref}}$. At $10\gamma_{\text{ref}}$ the training macro F1 reaches 1 for $C \geq 10$, while the validation score drops to about 0.865, so the model overfits. The best combination, $C = 10$ and $\gamma = 3\gamma_{\text{ref}} = 0.0232$, lies inside the grid.

Retrained on the full training set, the selected model reaches a training macro F1 of 0.982 and a



(a) Grid search

(b) Selected model

Figure 5: RBF SVM on the validation set. (a) Macro F1 for each combination of C and γ, trained on the 20% subset. (b) Row-normalized confusion matrix of the selected model ($C = 10$, $\gamma = 3\gamma_{\text{ref}}$), trained on the full training set.

validation macro F1 of 0.939, higher than the 0.914 obtained on the subset. Compared with logistic regression, the main difference is on class N: its recall increases from 0.73 to 0.996, so almost no normal beats are flagged as arrhythmias. As a consequence, the F1 of the minority classes improves considerably, from 0.33 to 0.89 for S and from 0.18 to 0.85 for F.

3.4 Fully connected neural networks

We implemented multilayer perceptrons (MLPs) using `MLPClassifier` from scikit-learn [2] to learn nonlinear relationships between heartbeat features and the five AAMI classes. The initial configuration used the 250 raw signal samples as inputs and two fully connected hidden layers of 100 neurons each. All configurations used ReLU hidden-layer activations, the Adam optimizer and a maximum of 200

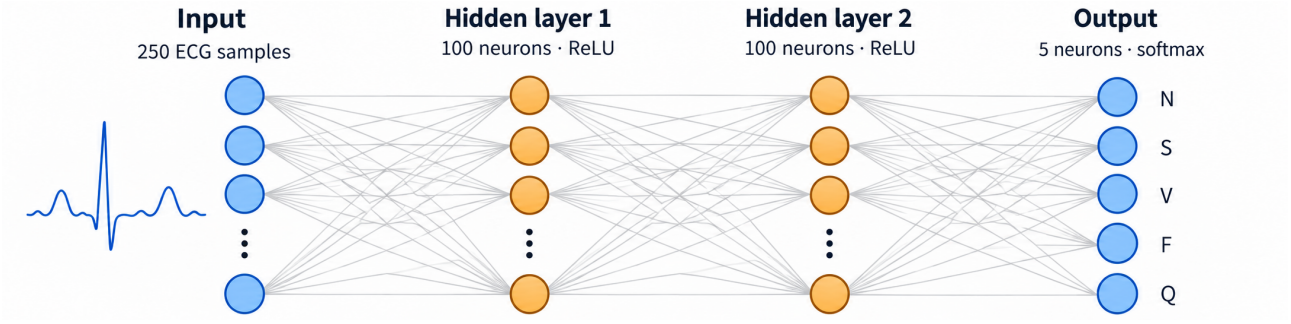


Figure 6: Selected MLP architecture for classification of raw ECG heartbeats.

training epochs. We fixed `random_state=42` for reproducibility. Training could terminate before this limit when the training loss no longer improved sufficiently, and this is what happened all the times.

Hyperparameter tuning consisted of targeted comparisons using validation macro F1. We tested one, two and three hidden layers with 100 neurons per layer. For the two-layer architecture, we also tested 50 and 200 neurons per layer. The L2 regularization coefficient α was increased from 10^{-4} to 10^{-3} to assess whether stronger regularization improved generalization.

The best tested configuration remained the initial two-layer network with 100 neurons per layer and $\alpha = 10^{-4}$, achieving a validation macro F1 of 0.9375 after 156 training epochs. This architecture was also evaluated with standardized, PCA and FFT features. None of the tested preprocessing configurations improved upon its performance on raw signals.

3.5 Convolutional neural networks

We implemented a one-dimensional CNN using TensorFlow/Keras [3] to investigate whether learning local waveform patterns could improve heartbeat classification. Raw signals were reshaped into sequences of 250 samples with one input channel. Convolutional filters and classification layers are trained jointly [4].

The architecture contains two convolutional layers with 16 and 32 filters, respectively, kernel size 5, ReLU activation and same padding. Each convolutional layer is followed by max pooling with pool size 2. The resulting feature maps are flattened and passed to a fully connected layer with 64 ReLU neurons, followed by five softmax output neurons.

Training used Adam with learning rate 10^{-3}, sparse categorical cross-entropy loss and batches of 128 heartbeats, for a maximum of 50 epochs. Early stopping monitored validation loss with a patience of five epochs and restored the weights corresponding to its minimum.

We compared the initial architecture with a variant adding dropout at rate 0.3 after the hidden dense layer, and a separate variant adding a second dense layer of 64 neurons. Configurations were selected using validation macro F1. The dropout variant achieved the best CNN score of 0.9273, remaining below the 0.9375 obtained by the selected MLP.

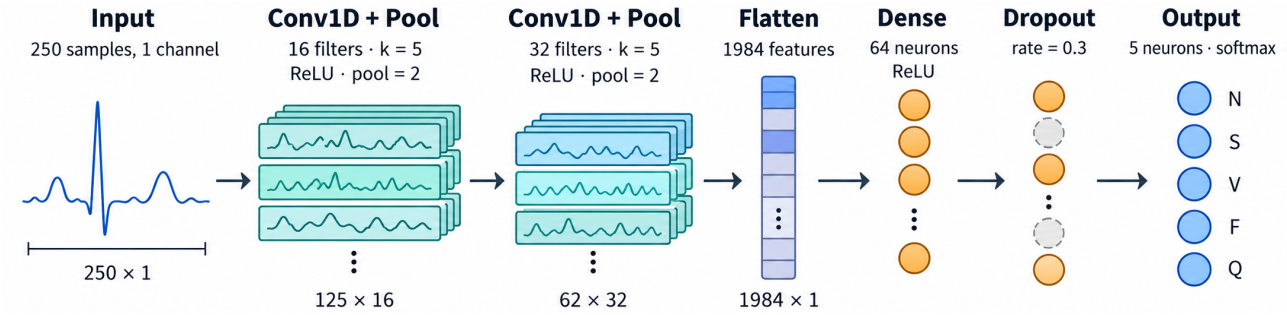


Figure 7: Selected CNN architecture for raw ECG classification. Each convolutional block includes ReLU activation and max pooling; k denotes the kernel size.

4 Results and Discussion

4.1 Validation results and hyperparameter selection

We compared 108 candidate configurations: 72 for logistic regression, 20 for RBF SVM, 13 for MLP and three for CNN. Preprocessing and hyperparameters were compared using macro F1 on the supplied validation set. SVM tuning used a stratified 20% subset of the training data; the selected configuration was then fitted on the full training set. Table 4 compares the best tested configurations with the majority baseline.

Model	Selected input representation	Val. macro F1
Majority baseline	–	0.181
Logistic regression	Per-beat normalization	0.554
RBF SVM	Per-beat normalization + PCA (100 components)	0.939
MLP	Raw signals	0.9375
CNN with dropout	Raw signals	0.9273

Table 4: Validation performance of the baseline and the best tested configuration of each classifier.

The selected SVM, MLP and CNN substantially outperformed logistic regression and were retained for final test evaluation.

4.2 Final test results and error analysis

The configurations selected using validation macro F1 were evaluated on the test set. Table 5 reports their final performance. The ordering observed on validation was preserved, with the SVM achieving the highest test macro F1, followed by the MLP and CNN. All three models obtained slightly lower macro F1 than on validation.

Model	Accuracy	Macro P	Macro R	Macro F1	Weighted F1
SVM	0.988	0.940	0.929	0.935	0.988
MLP	0.9864	0.9251	0.9301	0.9275	0.9865
CNN	0.9854	0.9622	0.8761	0.9134	0.9848

Table 5: Final test results for the selected configurations. P and R denote precision and recall.

Accuracy exceeded 98.5% for every model, while macro F1 revealed larger differences. This reflects the strong class imbalance: reliable classification of the majority class can coexist with poorer recognition of minority classes.

Table 6 shows the per-class F1 scores. All models performed well on N, V and Q, while S and F remained more difficult. The SVM's advantage over the MLP was most evident on S. Its precision for this class was 0.895, compared with 0.851 for the MLP, while their recalls were similar, at 0.888 and 0.892, respectively. On F, their F1 scores were almost identical.

Model	N	S	V	F	Q
SVM	0.994	0.892	0.973	0.820	0.994
MLP	0.9930	0.8709	0.9605	0.8190	0.9938
CNN	0.9920	0.8253	0.9682	0.7885	0.9928

Table 6: Test F1 scores for each AAMI class.

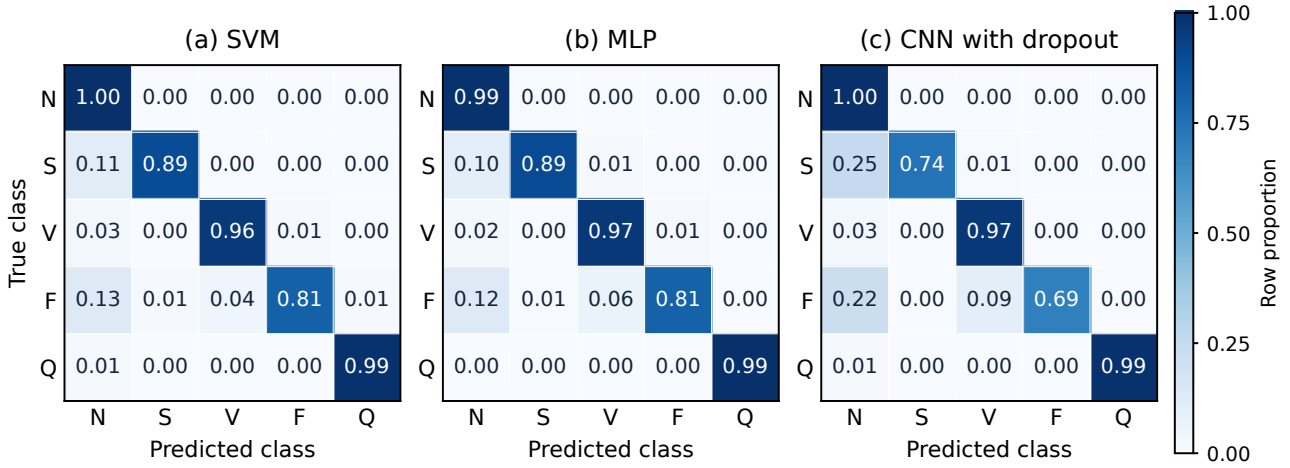


Figure 8: Row-normalized test confusion matrices for the selected SVM, MLP and CNN. Rows represent true classes and columns predicted classes. Values are proportions, rounded to two decimal places.

Figure 8 shows the distribution of classification errors. The CNN combined high macro precision with lower macro recall, particularly for S and F. It correctly identified 73.9% of S beats and 68.8% of F beats. Its confusion matrix shows that 141 of the 556 S beats were classified as N. Among the 160 F beats, 35 were classified as N and 15 as V. These errors explain why its high overall accuracy did not translate into equally strong performance across all classes.

4.3 Additional experiment: 15-class classification

We retrained the selected raw-signal MLP on the original 15 labels, retaining the same architecture and hyperparameters. Validation accuracy was 98.47%, with a macro F1 of 0.7140 across all 15 labels.

Label 9 has only two training examples and none in validation, so its generalization performance could not be assessed. This absent class contributes an F1 of zero to the reported 15-class average. Restricting the average to the 14 classes present in validation gives a macro F1 of 0.7650.

Labels 4 and 14 had only three and six validation examples, respectively, and no correct predictions. Their zero F1 scores show how overall accuracy can conceal poor recognition of rare classes.

4.4 Limitations

Experiments used the supplied fixed splits and a single random seed. Variability across data splits and neural network initializations was therefore not quantified, and small performance differences should be interpreted cautiously.

Class imbalance also limits the reliability of per-class estimates. For example, class F contains only 160 test beats, making its results sensitive to a small number of predictions. High aggregate scores consequently provide limited evidence of reliable recognition of every heartbeat type.

Hyperparameter searches covered finite grids and selected neural network architectures, with different search spaces for each model family. The SVM grid additionally used only 20% of the training data, so the selected parameters might differ from those preferred when tuning on the full training set. Our conclusions apply to the tested configurations and search spaces.

5 Conclusions

We compared logistic regression, RBF SVM, MLP and CNN models for ECG heartbeat classification into five AAMI classes, selecting configurations using validation macro F1.

The selected SVM achieved the highest test macro F1, approximately 0.935, followed by the raw-signal MLP at 0.9275 and the CNN at 0.9134. The MLP used two hidden layers of 100 neurons. Among the tested architectures, convolutional layers and greater network complexity did not improve upon this configuration.

Preprocessing choices depended on the classifier. The selected SVM used per-beat normalization and 100 PCA components, while raw signals remained the best MLP input. Real and imaginary FFT features gave similar validation performance to raw signals; magnitude-only features reduced dimensionality at the cost of lower macro F1.

Recognition of S and F remained challenging. The additional 15-class experiment further showed how high accuracy can conceal poor recognition of rare heartbeat types. Further work could quantify variability across random seeds and explore a broader range of CNN architectures.

References

[1] TU Delft. *EE4C12: Machine Learning for Electrical Engineering Applications*. Course lecture slides and code labs, including Feature Engineering and Learning, Neural Networks, and Machine Learning Workflow.

[2] scikit-learn developers. *scikit-learn documentation: User Guide and API Reference*. `https://scikit-learn.org/stable/`. Accessed 9 October 2026.

[3] TensorFlow developers. *TensorFlow documentation: Keras API*. `https://www.tensorflow.org/api_docs/python/tf/keras`. Accessed 4 October 2026.

[4] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. arXiv:2104.13478, 2021, Sections 4.2 and 5.1. `https://arxiv.org/abs/2104.13478`.

Use of AI. ChatGPT was used to support report drafting and summarization, help organize the results, and assist with writing and debugging selected Python notebook cells. AI was also used to create the NN and CNN architecture diagrams and adapt the layout of the PCA variance plot. The authors are responsible for the final content, analyses and conclusions.